Formulating Dr. Aura 🧬

An End-to-End Walkthrough of RAG Engineering over PubMed

Ehsan Abdi

4 March 2026

AuraQuery System Overview

System Elements	Implementation Details
Domain Focus	Biomedical Literature (PubMed)
Estimated Corpus Size	~3 Million Tokens (Abstracts + Full Body Texts)
Chunking Strategy	Semantic Section-Level (1000 max-tokens, 200 overlap)
Embedding Model	`text-embedding-3-small` (OpenAI)
Vector Database	Qdrant Cloud (Hybrid Search: Dense Vectors + Sparse BM25)
Retrieval Architecture	4-Stage Pipeline (Two-Stage Retrieval, Reranking, Diversity Control)
Primary LLM (Generator/Parser)	`gpt-4o-mini` (OpenAI)
Evaluation	LLM-as-a-Judge: `Llama-3.3-70b-versatile` (Groq)
Core Python Infrastructure	FastAPI Backend + Dockerized Environments

Navigating the PubMed Database

Building a RAG on a medical database introduces specific architectural and data processing considerations compared to standard document retrieval.

🔍 Inherent Characteristics

- **Terminology & Synonym Explosion:** Medical concepts have dense, overlapping vocabularies (e.g., “HHT,” “Osler–Weber–Rendu”); missing synonyms = missed evidence.
- **Evidence Quality & Conflicts:** Mixed study types (case reports → RCTs) with contradictory findings require ranking and context.
- **High-Stakes Hallucination Risk:** Small retrieval gaps can yield unsafe conclusions; strict grounding and citations are essential.

🎓 The RAG Learning Opportunity

- **Domain-Aware Retrieval:** Learn how query parsing, synonym normalization, and hybrid search (BM25 + embeddings) improve recall in terminology-dense domains.
- **Evidence-Aware Ranking:** Learn to rank by study type, recency, and clinical strength, and enforce structured citation grounding to reduce hallucinations.
- **Chunking Strategy:** Optimize chunk size and structure (abstract vs. section-level) to balance context, precision, and citation accuracy.

Data Ingestion & Preprocessing

The Dataset: Focused exclusively on Open-Access PMC articles retrieved via the **Biopython Entrez API**.

The "Two-Tier JSON" Strategy

To optimize retrieval efficiency, the ingested data was split into two discrete indices.

Tier 1 JSON: Abstract + Metadata
Utilized for fast, broad semantic filtering to establish relevance.

Tier 2 JSON: Full-Text + Metadata
Utilized for deep-dive paragraph extraction once relevant papers are identified.

✨ Data Transformation

- **Metadata Preservation:** Maintained key metadata such as *MeSH Keywords*, *Publication Type*, *publication year*, and *journal name* to enable rigid filtering during retrieval.
- **Recursive Indicators:** Stripped non-essential XML tags and applied markdown indicators (`##`) to demarcate section headers (e.g., `## Introduction`).

Chunking Strategy

Semantic Chunking Method

Documents were parsed natively using the structural headers (`##`) established during ingestion.

- **Abstracts:** Maintained as a single, holistic chunk.
- **Full-Text:** Chunked strictly per section (e.g., Methods vs. Results).
- **Length Control:** If a section exceeded the optimal length, a fallback of **1000 max-tokens with a 200-token overlap** was applied.

Index Size Comparison

Each chunk inherited the parent document's metadata, alongside an added `section_name` field.

Index Layer	Item Count
Index A (Abstracts)	~888 documents
Index B (Body Chunks)	~16,972 chunks

Choosing the Embedding Model

Translating complex biomedicine into mathematical vector representations.

Model Option	Type	Clinical Reasoning	Cost / Accessibility
OpenAI `text-emb-3-small`	Dense	Offers strong general semantic understanding, though may exhibit slightly lower precision on rare acronyms.	Highly cost-effective and accessible.
PubMedBERT	Dense	Highly specific domain pre-training for biomedical text.	Open source, but strictly limited by a small 512-token context window.
MedEIR	Dense	Specialized medical embeddings, supports long context windows.	High compute cost and limited testing on rare or new medical terms.

🏆 **The Choice: OpenAI & FastEmbed**

For the scope of this project, **OpenAI** was prioritized for dense vectors given its balance of performance and efficiency. Additionally, **FastEmbed** was utilized for sparse vectors to enable native BM25 keyword search.

The Vector Database Deployment

Initial phase development commenced with a localized **ChromaDB** instance before transitioning to cloud infrastructure.

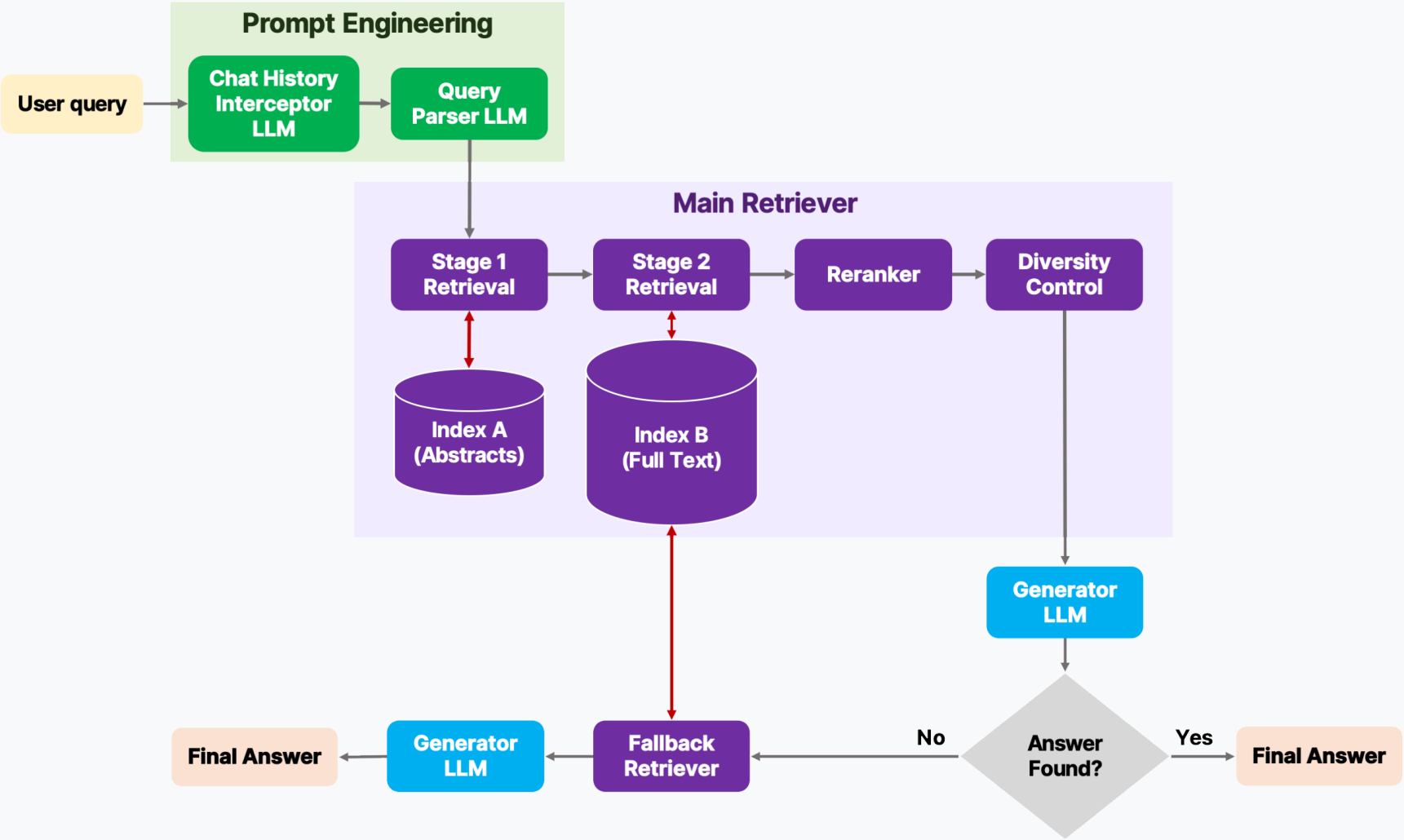
Advantages of Local DB

- Zero infrastructure cost and immediate setup.
- Operates entirely within the Python runtime environment.
- Ideal for rapid prototyping and offline algorithmic development.

Limitations Encountered

- **No Native Hybrid Search:** Out-of-the-box, ChromaDB lacked combined dense vector and BM25 keyword search functionality.
- Because medical queries require exact keyword matching, a manual two-stage Python process had to be engineered.

Two-Stage Semantic Retrieval Strategy



Prompt Engineering: Stage 1

Introducing chat history complicates standard RAG retrieval (e.g., resolving pronouns).

The Interceptor (Chat History Parser)

Model: `gpt-4o-mini` — *A lightweight, high-speed LLM for rapid conversational pre-processing.*
Configuration: `temperature=0.0` (Strict Determinism)

- **Role:** An LLM is tasked with reading the **chat history**, **resolving pronouns**, and extracting **explicit identifiers** from previous AI responses into the new query string.
- **Why it matters:** Vector databases cannot "remember" previous chat context. A query like *"What about its side effects?"* will fail a vector search.
- **Example:**
  - **User:** *"What is the standard dosage?"*
  - **Interceptor ➔** *"What is the standard dosage for Bevacizumab? PMID: 1234567"*

Prompt Engineering: Stage 2

The Query Parser (Query Optimization & Metadata Extraction)

Model: `gpt-4o-mini`
Configuration: `temperature=0.0` (Strict Determinism)

Once the query is contextually resolved, a second lightweight LLM chain prepares it for the Hybrid Search engine. It executes 5 core operations:

1. **Clarify and Standardize:** Converts layperson phrases to clinical terms.
2. **Controlled Synonym Expansion:** Adds high-value synonyms without keyword stuffing.
3. **Chunk-Aware Optimization:** Strips conversational filler to keep the query tight.
4. **Ambiguity Detection:** Requests user clarification if a term is hopelessly ambiguous.
5. **Metadata Extraction:** Isolates structured filters using Pydantic models.
    - **Example:** *"Show me trials from 2024 by Shovlin"* ➔ Extracted Filter: `{ "year": 2024, "author": "Shovlin" }`

The 4-Stage Retrieval Pipeline

To combat information overload and ensure high-precision grounding, the retrieval architecture operates in four distinct phases:

🔍 1. Abstract Discovery (Index A)

A hybrid search over full article abstracts to quickly identify the top 50 most relevant candidate papers.

🎯 2. Deep Chunk Search (Index B)

A targeted dense search over the specific body paragraphs of ONLY those 50 candidate papers.

⚖️ 3. Algorithmic Reranking

Adjusting raw vector scores based on methodology strength (e.g., RCTs), recency, and section importance.

🛡️ 4. Diversity Control

Hard-capping chunks per PMID to prevent a single review article from monopolizing the LLM's context window.

Stage 1: Abstract Discovery

Initial broad context retrieval to identify candidate papers.

🔍 Process Overview

- Runs a **hybrid search (Dense + Sparse BM25)** across **Index A (Abstracts)**.
- Applies any strict **metadata filters** extracted earlier (e.g., Publication Year, Authors).
- Selects the **top 50 unique PubMed IDs (PMIDs)** that are most conceptually aligned with the query.

Stage 2: Deep Chunk Search

Drilling down into the specific body paragraphs of identified candidate papers.

🎯 Targeted Retrieval

- Executes a **hybrid search (Dense + Sparse BM25)** across **Index B (Full-Text Body Chunks)**.
- Restricts the search space strictly to the **50 Candidate PMIDs found in Stage 1** using a `MatchAny` filter.
- Pulls the **top 80 most relevant paragraphs/chunks** for deeper analysis.

Note: If no candidates are found, a global fallback search on Index B is automatically triggered.

Stage 3: Algorithmic Reranking

Vector similarity is not enough. We manually adjust scores based on clinical quality and relevance.

Scoring Criteria	Implementation Details	Score Adjustment
Methodology	Prioritizes highest evidence quality.	Meta-Analysis (+1.0), RCTs (+0.85), Case Reports (+0.5)
Section Bias	Favors key informational sections over filler.	Results/Conclusion (+1.5), Methods (+1.0), Intro (-0.5)
Recency	Exponential decay curve based on publish year.	`0.25 * exp(-diff / 8)`

Mechanism: The base vector similarity score from Qdrant is summed with the adjustments above. The final set of chunks is re-sorted based on this new holistic score.

Stage 4: Diversity Control

Ensuring the LLM context window represents a synthesis of multiple papers.

🛡️ The Problem: Monopolization

Without intervention, a **single highly-relevant review article** might populate a significant portion of the returned chunks, drowning out novel primary research.

✅ The Solution: Hard-Capping

- The system enforces a strict maximum of **5 chunks per individual PMID**.
- Truncates the final curated list of varied chunks to the target return size (e.g., **top 30**) before injection into the prompt.

Global Fallback Search

What happens when the 4-stage retrieval pipeline fails to provide the Generator LLM with the answer?

🚨 The Trigger

After the initial Pipeline retrieval, the Generator LLM attempts to answer. If it concludes the evidence is insufficient, it safely refuses to answer and drops an internal flag.

🔄 Bypassing the Candidates

- The system automatically intercepts the failure flag.
- It entirely **bypasses Index A** and executes a broad **Hybrid Global Search** (Dense + BM25) directly across the entirety of **Index B (Body Chunks)**.
- This guarantees that hyper-specific findings missed in a paper's abstract can still be retrieved directly from within a paper's body text.

The Generator LLM

Following context assembly, a generative model synthezises the final response.

The LLM Selection: `gpt-4o-mini`

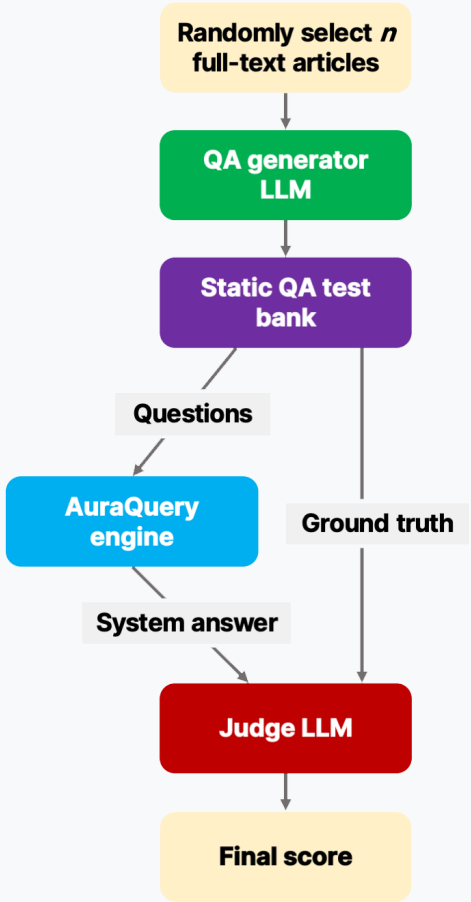
- It provides rapid inference times, is highly cost-effective, and maintains sophisticated instruction-following capabilities relative to its parameter size.

Temperature Specification (0.0)
In biomedical contexts, variance and creativity equate to hallucination risk. Setting the temperature to 0.0 enforces deterministic outputs, anchoring the LLM strictly to the provided evidentiary chunks.

Strict Grounding Constraint
The system prompt strictly forbids external knowledge. If the generator is unable to find an answer explicitly grounded in the retrieved documents, it is forced to generate an exact flag: ***"I couldn't find sufficient evidence in the literature to answer this question."***

Evaluation: Dual LLM-as-a-Judge

Quantitative validation was performed using an autonomous framework where one LLM family (OpenAI) generates outputs and another (Groq) judges them, structurally eliminating self-preference bias.



The Asynchronous Framework

- **Step 1 (Generation):** An LLM (`gpt-4o-mini`) extracts ground-truth facts from chunks and generates structurally diverse questions into a static test bank JSON.
- **Step 2 (Execution):** The Aura pipeline retrieves context and generates an answer without knowledge of the ground truth.
- **Step 3 (Judging):** A separate model family (`Llama-3.3-70b-versatile` via Groq) scores the final answer against the hidden ground truth, preventing self-preference bias.

System-Level Evaluation Outcomes

Evaluating the end-to-end performance of the AuraQuery pipeline across 85 static research questions.

AVERAGE SYSTEM SCORE: 9.19 / 10.00

Total Questions Evaluated:	85
Average Response Latency:	13.39 seconds
Perfect Scores (10/10):	48 (56.5%)
Zero Scores / Failures:	2 (2.4%)

Cloud Migration: Entering Production

The system transitioned from a local ChromaDB instance to a managed **Qdrant Cloud** database.

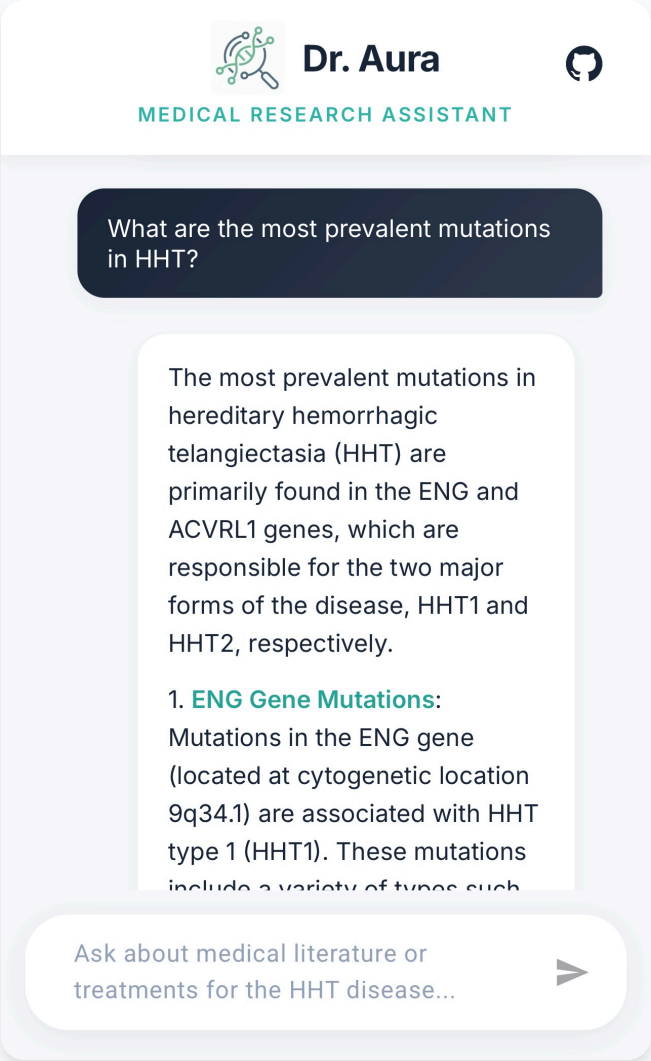
Vector DB Option	Advantages	Disadvantages	Verdict
Qdrant Cloud	Generous free tier and native support for hybrid (Dense + BM25) search out of the box.	Minor integration adjustments required for LangChain compatibility.	🏆 **Selected.** Ideal free tier for experimental projects while supporting essential hybrid search capabilities.
Pinecone	Industry standard, robust APIs.	Proprietary nature and restrictive free-tier limitations.	Not prioritised for this phase.

*This migration significantly **reduced inference latency** and **decoupled the data layer** from the application compute environment.*

Frontend & Containerization

Deployment Infrastructure

- **Frontend Interface:** Designed around a modern chat-based UI using frameworks applicable for stateful conversations.
- **Backend Excision:** The retrieval engine was decoupled and exposed via a robust **FastAPI** REST endpoint.
- **Containerization:** The application environments were encapsulated within **Docker containers** to guarantee environment parity across deployments.



Next Steps and Potential Improvements

⚙️ Evaluation & Optimization

- **Define Metrics:** Establish component and system-level evaluation metrics (e.g., latency, throughput, memory usage, faithfulness, response relevancy).
- **Integrate Phoenix:** Add Arize Phoenix in order to perform detailed tracing, optimization, testing, and continuous evaluation.
- **Component Optimization:** Optimize chunking strategy, embedding model, query parser, index A & B retrieval, reranker, diversity control, and choice of generator LLM.
- **System Optimization:** Optimize the system architecture balancing **cost** vs **latency** vs **accuracy**.
- **Scale Up Corpus:** Increase the size of the corpus from ~3M tokens to ~10M tokens to assess and further optimize retriever performance at scale.

Thank You! 🚀